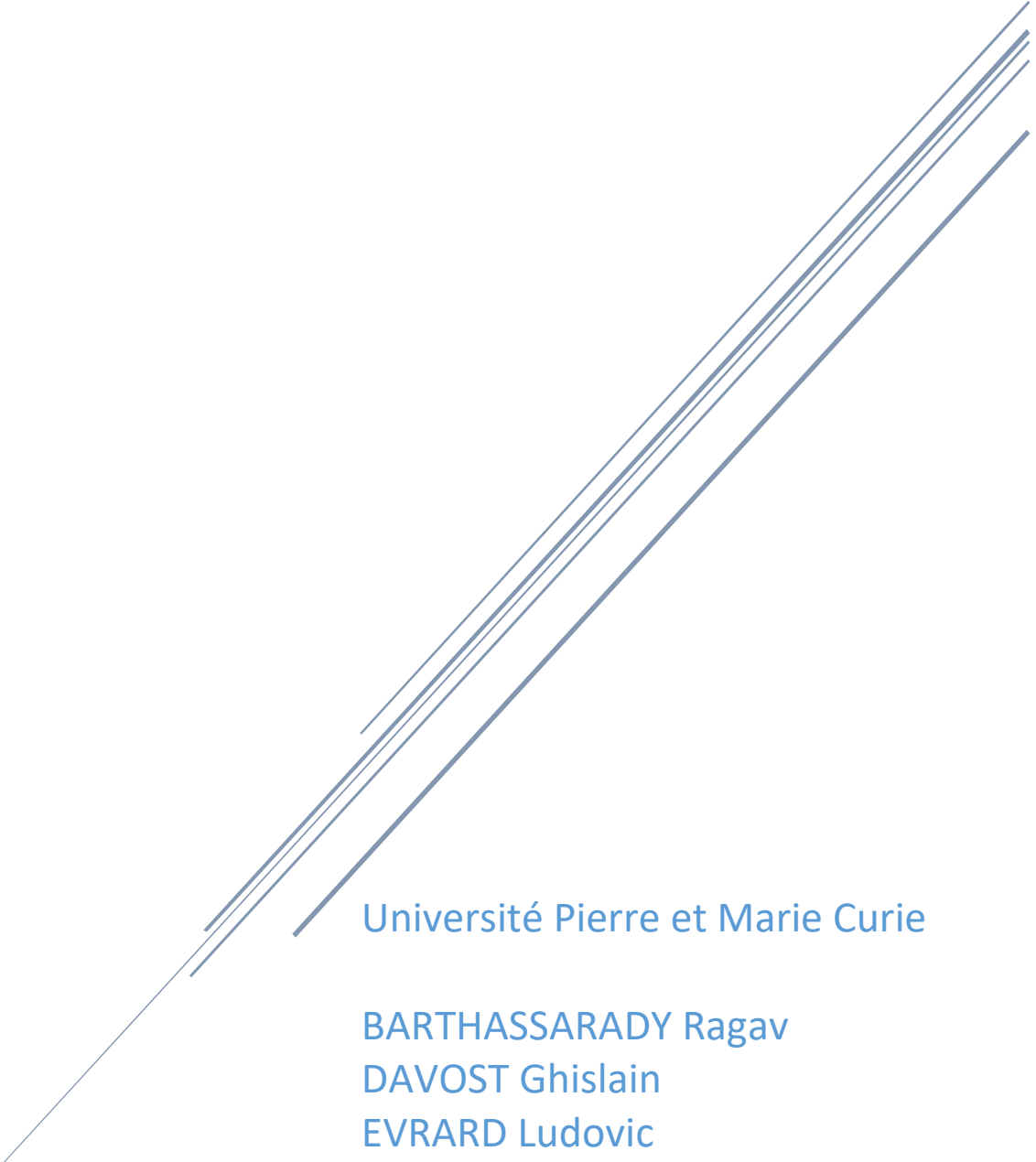


PROJET ELECTRONIQUE - INFORMATIQUE

Chaine de production alimentaire contrôlée par MCU



Université Pierre et Marie Curie

BARTHASSARADY Ragav

DAVOST Ghislain

EVARD Ludovic

VINAYAGAMOORTHY Vinoth

Table des matières

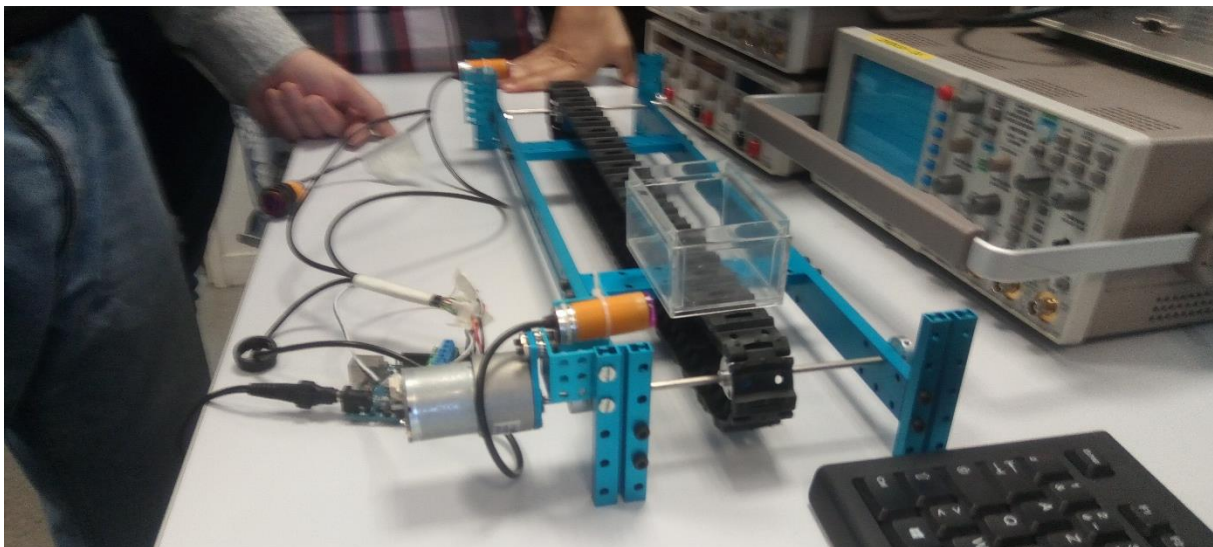
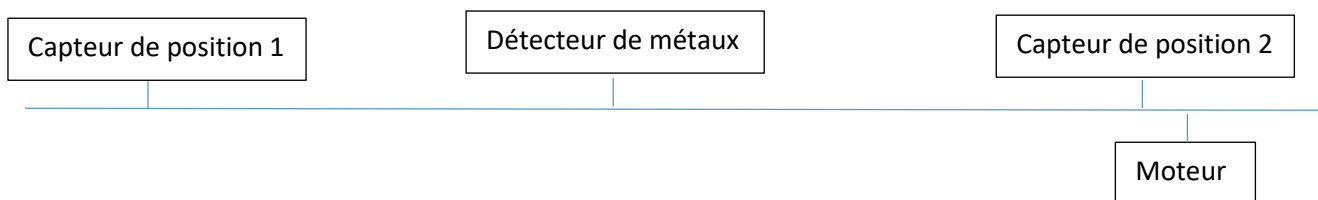
Table des matières	1
Introduction.....	2
Description	2
1. Partie informatique	2
Explication du fichier serialtransfert	3
Explication de l'implémentation de la machine virtuelle.....	3
Explication du code :	3
Explication du programme	4
2. Partie électronique.....	5
Le moteur	6
Solution choisie pour faire fonctionner le moteur	6
Le détecteur de métaux	8
Solution choisie pour la création de ce détecteur	8
Calcul de la capacité pour une fréquence de résonance du module de réception de 300 kHz	8
Test des bobines sur l'analyseur de spectre.....	9
Fonctionnement du capteur.....	9
Test de fonctionnement	9
Conclusion sur l'efficacité du capteur	12
Capteur de présence	12
Planning de réalisation du projet électronique.....	13
Taches capteur de présence.....	13
Conclusion	14

Introduction

Le but du projet informatique - électronique est de créer un banc permettant de simuler les tapis automatisés qu'on peut trouver dans l'industrie de l'agro-alimentaire avec capteurs nécessaires au bon fonctionnement du système. Le système est programmé en grafcet. Le but de la partie informatique est de réaliser un compilateur de grafcet qui sera traité sur un microcontrôleur. Le microcontrôleur fait l'interface informatique/électronique, gère tous les signaux. Le système se décompose en un tapis roulant propulsé par un moteur et 2 capteurs de position au début et en fin de tapis. Un système permettant de repérer le métal sera disposé au centre.

Description

Pour mieux comprendre comment doit fonctionner le tapis automatisé, nous avons créé le schéma ci-dessous :



Ce tapis doit être contrôlé à l'aide d'une machine virtuelle implanté sur un microcontrôleur STM32.

1. Partie informatique

Le code est commenté méthodiquement pour plus de clarté et qu'il soit facilement repris par la suite. L'organisation à suivre nous a été fourni par M. Griot, elle est la suivante :

```
remi@remi-VirtualBox:~/Desktop/Dropbox/Work/UPMC/ProjetInfoElec/ProjetInfoElec-E
I2I3$ tree
.
├── asm
│   ├── asm.c
│   └── asm.h
├── ast
│   ├── ast.c
│   └── ast.h
├── bison
│   ├── grafcet.y
│   ├── y.dot
│   ├── y.tab.c
│   ├── y.tab.h
│   └── y.tab.h.gch
├── build
├── graf
│   ├── basic2.grafcet
│   ├── basic.grafcet
│   ├── ex1.grafcet
│   ├── ex2.grafcet
│   └── safe.grafcet
├── grafInterpreter
│   └── graphInterpreter.c
├── lex
│   ├── grafcet.l
│   └── lex.yy.c
├── Makefile
├── ReadMe.pdf
├── vm
│   ├── evm.c
│   └── vm_codops.h
```

Le schéma de principe est le suivant :



[Explication du fichier serialtransfert](#)

Voir directement fichier .c commenté.

[Explication de l'implémentation de la machine virtuelle](#)

Rappel : La machine virtuelle doit permettre de manipuler les entrées-sortie du banc de test. Elle contient le programme permettant de gérer ces entrées-sorties (capteurs, moteurs etc). Nous avons déjà créé une machine virtuelle durant les séances de projet informatique. Afin de l'utiliser pour le projet électronique-informatique, nous devons implémenter ce code dans le microcontrôleur puis le tester.

[Explication du code :](#)

La machine virtuelle est composée de 3 tableaux :

1. Le tableau code qui contient le programme en dur ;
2. Le tableau var qui contient les valeurs des entrees sorties ;
3. Le tableau stack qui permet d'effectuer des operations.

Afin de pouvoir manipuler ces tableaux dans le main et dans les fonctions, nous avons déclaré ces tableaux en globale.

Afin de pouvoir parcourir les instructions dans les tableaux, nous avons créé deux variables :

1. pc, qui permet de parcourir le tableau code ;
2. sp, qui permet.

Dans le tableau code, on utilise plusieurs instructions. Ces intructions sont codés dans la fonction run :

```
void run(void){
    while(code[pc] != I_HALT){
        switch(code[pc]){
            case I_PUSHI : // permet de mettre une valeur du tableau code dans la stack
                sp++;
                stack[sp] = code[pc+1];
                pc = pc+2;
                break;
            case I_MULT : // permet de multiplier les deux valeurs contenues l'un à la suite de l'autre dans la stack
                stack[sp-1] = stack[sp-1] * stack[sp];
                sp--;
                pc++;
                break;
            case I_ADD : // permet d'additionner les deux valeurs contenues l'un à la suite de l'autre dans la stack
                stack[sp-1] = stack[sp-1] + stack[sp];
                sp--;
                pc++;
                break;
            case I_PUSH : // permet de mettre la valeur contenue dans une variable du tableau var dans la stack
                sp++;
                stack[sp] = var[code[pc+1]];
                pc = pc+2;
                break;
            case I_POP : // permet de mettre une valeur contenue dans la stack dans une variable du tableau var
                var[code[pc+1]] = stack[sp];
                sp--;
                pc = pc+2;
                break;
            case I_EQ : // on effectue la comparaison entre deux valeurs de la stack
                if(stack[sp]==stack[sp-1]) stack[sp-1]=1;
                else stack[sp-1]=0;
                sp--;
                pc++;
                break;
            else{
                pc = pc + 2;
                sp--;
            }
            break;
            case I_AND : // on fait une comparaison en et logique
                stack[sp-1] = stack[sp-1] && stack[sp];
                sp--;
                pc++;
        }
    }
}
```

L'implémentation de la machine virtuelle dans le programme du microcontrôleur s'est fait en plusieurs étapes :

- On copie la fonction run() dans le programme c ;
- On met en lien le fichier vm_codops.h avec le projet du microcontrôleur ;
- Puis on crée un petit programme permettant de manipuler les entrées sortie afin de valider l'implémentation de la machine virtuelle dans le programme.

Les deux premières étapes ont été effectuées très facilement.

Explication du programme

Le but du programme est de manipuler des entrées (PC8 ET PC9) en faisant une opération logique afin de gérer une sortie (PA5) du microcontrôleur.

Pour cela, nous avons donné des espaces dédiés à ces 3 pins dans le tableau var[]. Afin d'organiser le programme nous avons défini les indices des espaces mémoires dédiées à chaque patte directement à ces pattes à l'aide de l'instruction define :

```
#define PC8 0 // PC8 à l'emplacement var[0]
#define PC9 1 // PC9 à l'emplacement var[1]
#define PA5 2 // PA5 à l'emplacement var[1]
```

Puis dans le main , nous effectuons l'initialisation des pattes et des variables nécessaires à la lecture ou écriture sur chacune des pattes :

```
initPattePC8_entree();
initPattePC9_entree();
initPattePA5_LedVerte();
int entree1; // variable utilisée pour la lecture de PC8
int entree2; // variable utilisée pour la lecture de PC9
int sortie;   // variable utilisée pour écrire dans PA5
```

Puis on fixe à zéro le tableau var (permet d'éviter d'avoir des erreurs de lecture des entrées au début du programme) :

```
var[PC8]=0;
var[PC9]=0;
```

Puis on écrit le programme en dur dans le tableau code[] :

```
code[0]= I_PUSH;
code[1]= PC8; // on met la valeur contenue dans PC8 dans la stack
code[2]= I_PUSH;
code[3]= PC9; // on met la valeur contenue dans PC9 dans la stack
code[4]= I_OR; // on effectue un ou logique
code[5]= I_POP ;
code[6]= PA5; // on met le résultat dans PA5
code[7]= I_HALT; // on sort de la fonction run
```

Puis on entre dans le while(1).

```
while(1)
{
    entree1=((GPIOC -> IDR & 0x0100)>>8);
    entree2=((GPIOC -> IDR & 0x0200)>>9);
    var[PC8]= entree1;
    var[PC9]= entree2;
    pc=0;
    run();
    sortie=var[PA5];
    etat_led(sortie);
}
```

On lit les valeurs des entrées (PC8 et PC9)

On met ces valeurs dans les emplacements dédiés dans le tableau var[]

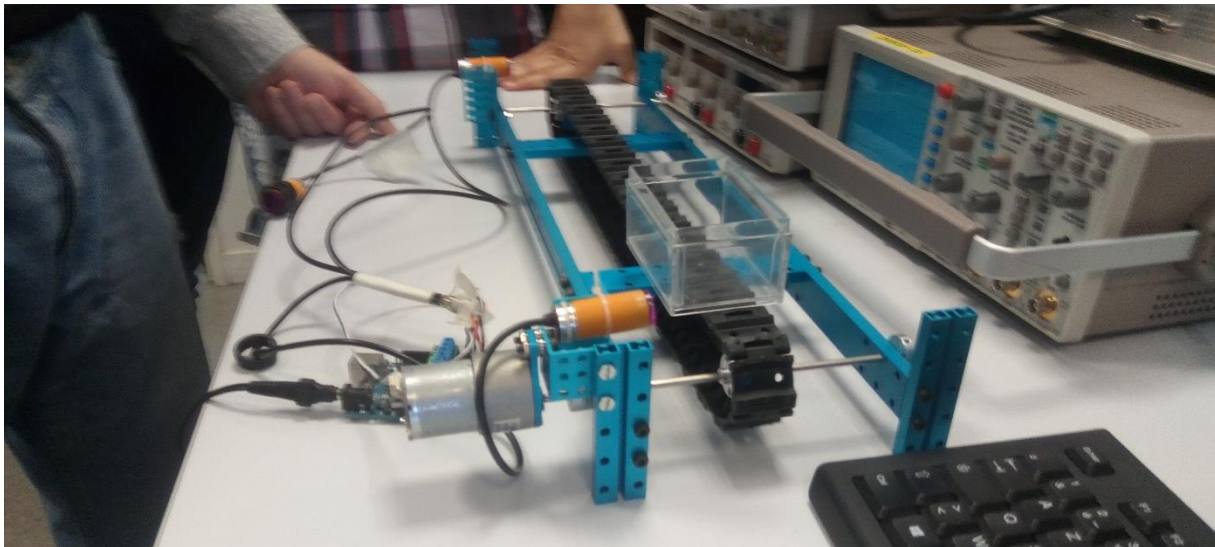
On lance la fonction run() qui permet de gérer le programme

On assigne la valeur contenue dans var[PA5] dans sortie

Puis on met la valeur contenue dans la variable sortie dans PA5

Les autres parties du code du projet sont expliqués directement en commentaire.

2. Partie électronique



Voici le système de tapis automatisé sur lequel nous travaillons

Le système est constitué des composants électroniques suivant :

- Un moteur, qui nous permettra de faire avancer le tapis dans les deux sens.
- Deux détecteurs de présences à chaque extrémité du tapis.
- Un détecteur de métal situé en milieu de tapis.

Grâce au microcontrôleur STM32 Nucléo nous devront faire fonctionner ces composants afin que le système soit capable de réaliser le cycle suivant :

- Un bac est posé en début de tapis, le premier détecteur de présence le détecte et fait avancer le tapis vers l'avant.
- Le bac atteint le milieu du tapis et passe à travers le détecteur de métal :
 - S'il y a détection de métal, le tapis fait marche arrière jusqu'à que le bac atteigne le premier détecteur et s'arrête afin que l'on puisse retirer le métal.
 - S'il n'y a pas de métal détecté le tapis continu de rouler vers l'avant.
- Lorsque le bac atteint le deuxième détecteur de présence en bout de tapis, ce dernier s'arrête afin que le bac puisse être déchargé puis à nouveau déposer devant le deuxième détecteur de présence afin qu'il retourne en début de tapis où il va s'arrêter au premier détecteur.

Etudions de plus près c'est composant ainsi que leur implémentation au sein du système.

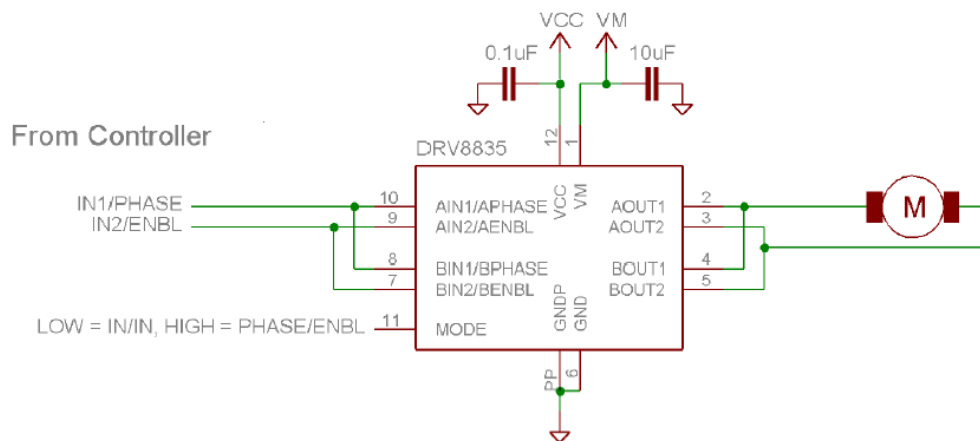
Le moteur

Le moteur est un des éléments principaux du système, il permet de faire défiler le tapis ainsi que les éléments présents sur ce dernier.

Le moteur devra fonctionner dans les deux sens, la marche avant pour mettre les aliments dans les bacs et la marche arrière afin de revenir en arrière lorsqu'un métal est détecté. Le moteur utilisé est un moteur à courant continu donc il faut déterminer comment faire fonctionner le moteur en marche avant puis en marche arrière.

Solution choisie pour faire fonctionner le moteur

On utilise un driver, le drv8835. En entrée de ce driver on connecte trois pins du microcontrôleur (utilisé pour contrôler le banc) et en sortie, du driver, on connecte le moteur :



Nous avons câblé notre driver en suivant ce schéma.

Table 2. IN/IN MODE

MODE	xIN1	xIN2	xOUT1	xOUT2	FUNCTION (DC MOTOR)
0	0	0	Z	Z	Coast
0	0	1	L	H	Reverse
0	1	0	H	L	Forward
0	1	1	L	L	Brake

Table 3. PHASE/ENABLE MODE

MODE	xENABLE	xPHASE	xOUT1	xOUT2	FUNCTION (DC MOTOR)
1	0	X	L	L	Brake
1	1	1	L	H	Reverse
1	1	0	H	L	Forward

Voici le tableau expliquant le fonctionnement du DRV en mode marche avant et marche arrière.

En suivant cette table de vérité nous avons conçu des variables permettant de faire avancer et reculer notre moteur selon différent rapport cyclique (25% ; 50% ; 90%).



Voici le moteur présent sur notre système.

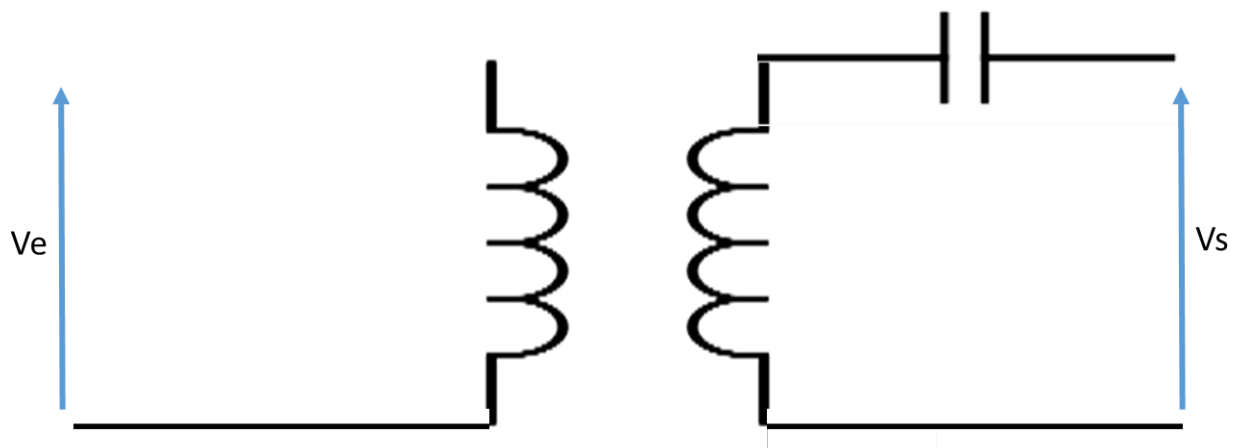
Le détecteur de métaux

Ce détecteur doit nous permettre de détecter tous types de métaux et de les différencier des aliments standards qui défilent sur la chaîne de production.

Solution choisie pour la création de ce détecteur

Le circuit d'émission est composé d'une résistance qui limite le courant dans la bobine émettrice. Le circuit de réception LC est quant à lui composé d'une capacité qui permet de fixer la fréquence de résonance du circuit.

$$f_0 = \frac{\omega_0}{2\pi} = \frac{1}{2\pi\sqrt{LC}}$$



Calcul de la capacité pour une fréquence de résonance du module de réception de 300 kHz

Nous souhaitons dimensionner notre système de détection avec une fréquence de 300 kHz

$$f_0 = 300 * 10^3 = \frac{1}{2\pi\sqrt{LC}}$$

$$\Leftrightarrow \frac{1}{f_0} = 2\pi\sqrt{LC}$$

$$\Leftrightarrow \frac{1}{2\pi * f_0} = \sqrt{LC}$$

$$\Leftrightarrow \frac{1}{L(2\pi * f_0)^2} = C$$

$$\Leftrightarrow \frac{1}{25 * 10^{-6}(2\pi * 300 * 10^3)^2} = C \approx 1 \text{ nF (valeur normalisée)}$$

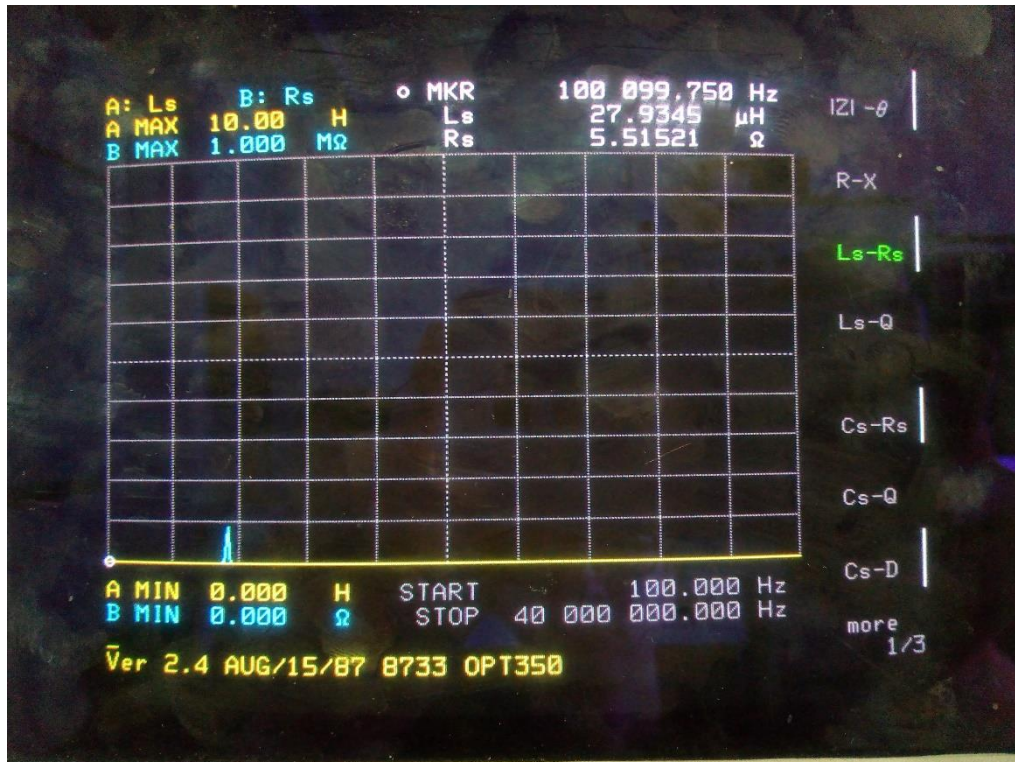
Nous avons donc pris un condensateur de 1nF.

Test des bobines sur l'analyseur de spectre

Nous avons utilisé une bouteille de plastique vide comme support pour faire des bobines. Avec du fil de cuivre nous avons fait 12 tours pour chacune des bobines.

La bobine d'émission à une inductance de 26 μH .

La bobine d'émission à une inductance de 28 μH .



Fonctionnement du capteur

Un générateur alimente en tension alternative (à la fréquence propre) le circuit RL série d'émission. Celui-ci capte transforme ce signal reçu en champs électrique.

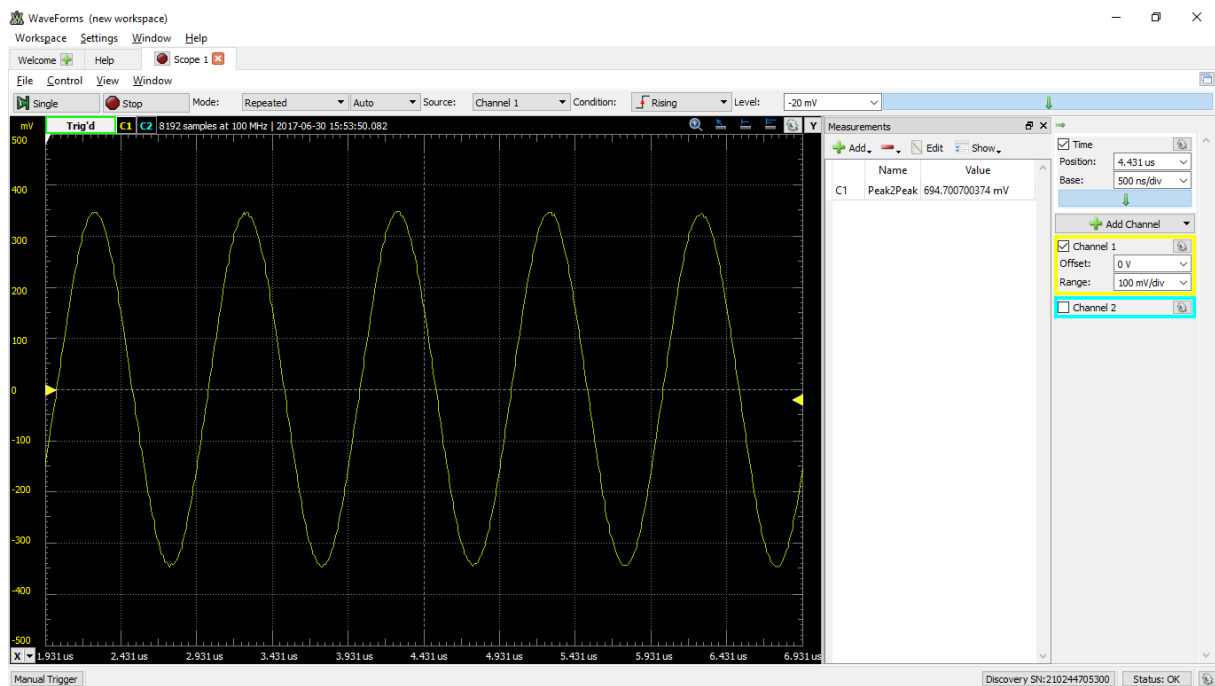
Ce champs électrique sera capté et transformé en tension alternative par le circuit LC série de réception et connecté à une pin du MCU STM32 configuré en ADC.

Lors du passage d'un métal, la fréquence reçue en réception restera là même, mais l'amplitude du système reçu change. La valeur reçue sur le MCU par le convertisseur analogique numérique est lue par une fonction que nous avons créée pour qu'elle différencie le signal standard (sans métal) et un signal différents du signal standard (indique la présence d'un métal). Le résultat de cette fonction est ensuite mis dans l'interruption pour qu'en cas de présence d'un métal le système de chaine de production puisse réagir en conséquence pour éviter un aliment contenant du métal d'être livré au client.

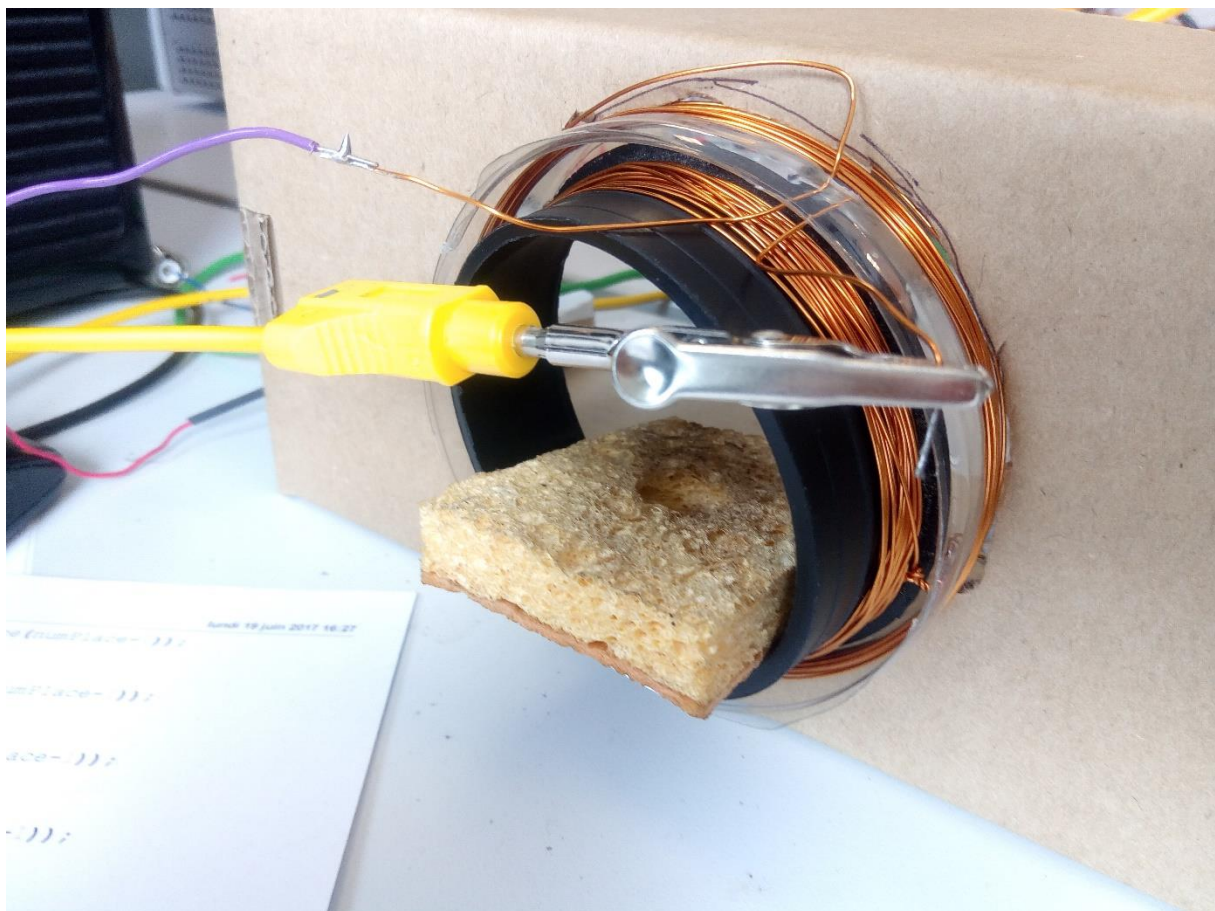
Test de fonctionnement

Nous avons testé notre système de détection pour vérifier s'il correspondait bien à nos attentes.

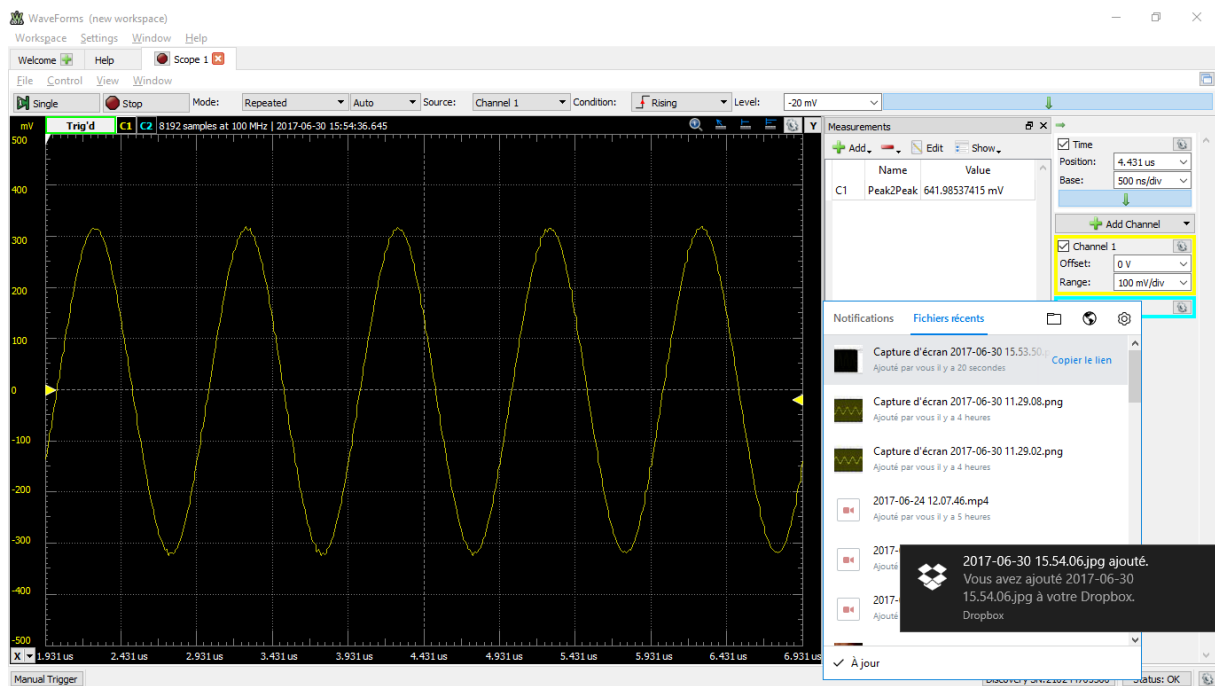
Dans le cas de nos essais l'objet (correspondant à l'aliment est sur la chaine de production) passe à travers les deux bobines.



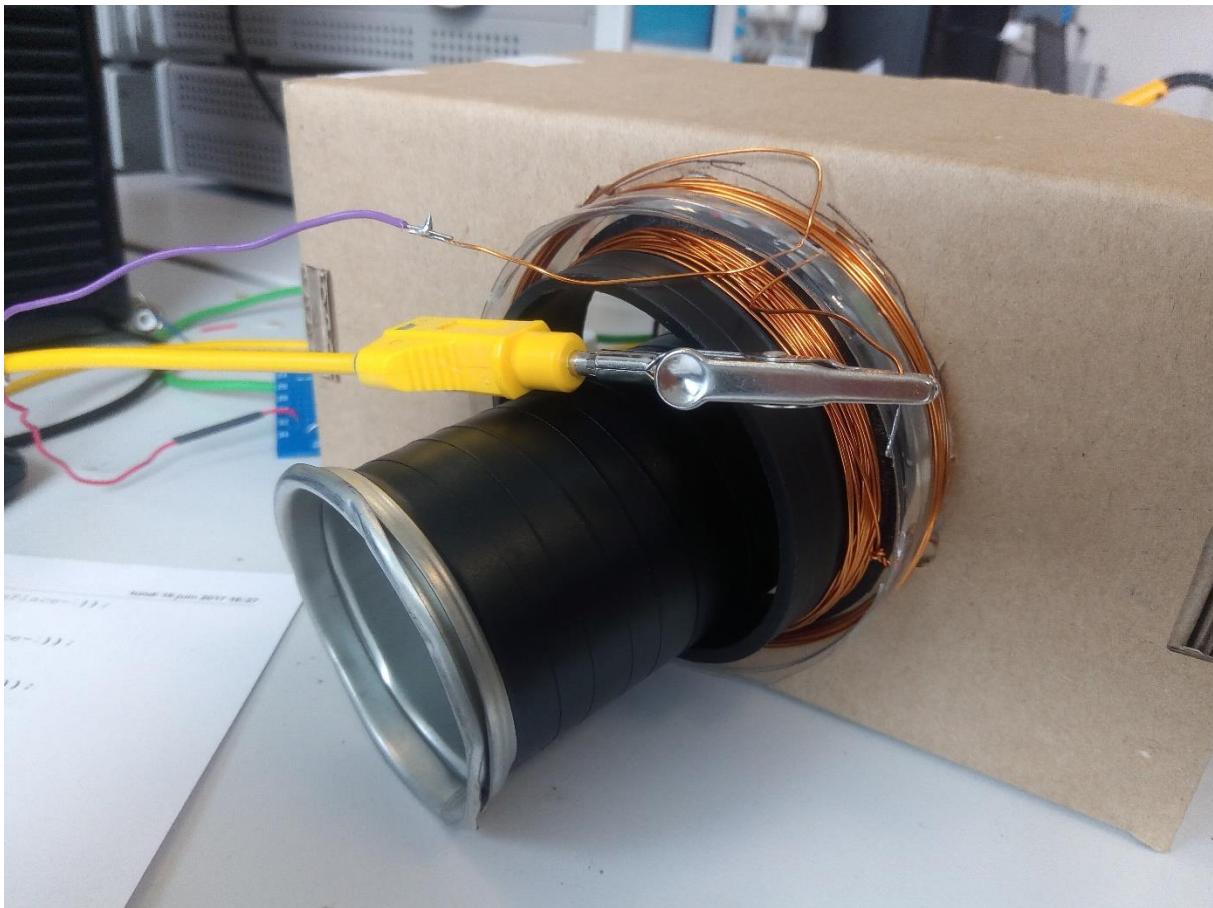
Capture du signal reçu sur la pin CAN du MCU sans objet



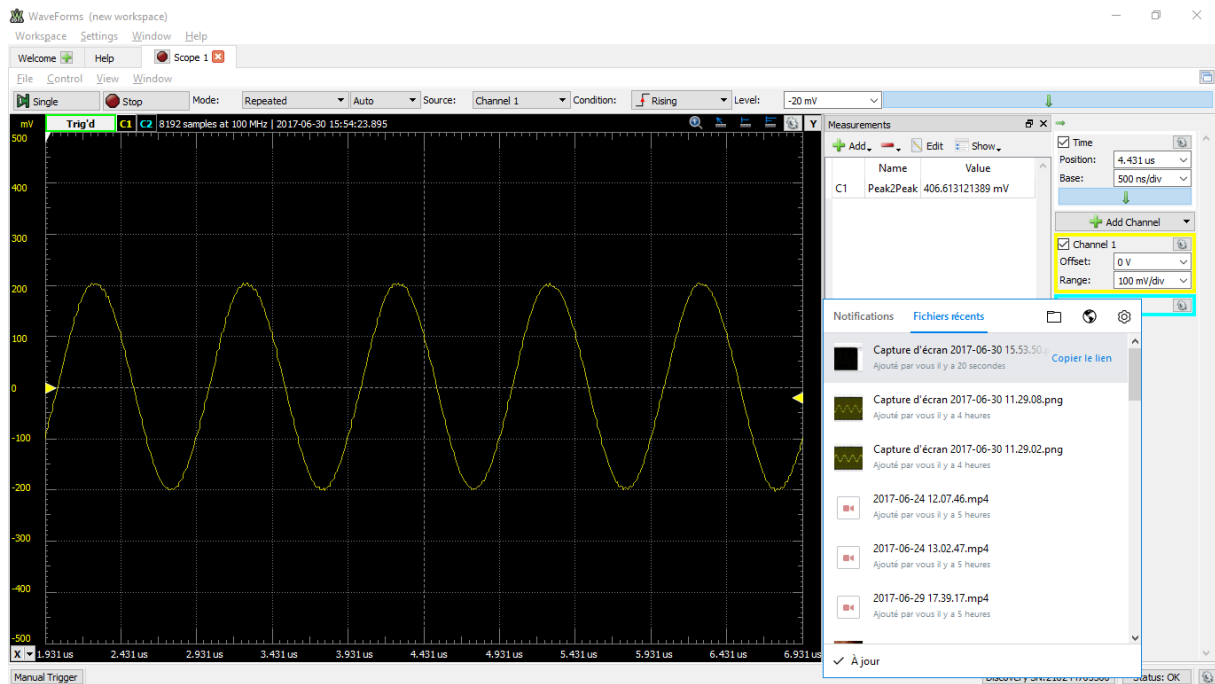
Passage de l'aliment conforme simulé sur la chaine de production



Capture du signal reçu sur la pin CAN du MCU lors du passage d'une éponge (aliment conforme sans métal)



Passage de l'aliment non-conforme simulé sur la chaîne de production



Capture du signal reçu sur la pin CAN du MCU lors du passage d'un cylindre métallique (aliment non-conforme)

Conclusion sur l'efficacité du capteur

On constate une diminution de plus de 40% $[(1-(0,694-0,406))*100]$ de l'amplitude du signal reçu lors du passage du cylindre métallique, tandis que lors du passage de l'éponge le signal reçu reste quasi inchangé.

Le capteur fonctionne donc de manière satisfaisante. Nous proposons de fixer un seuil de variation de l'amplitude de -10% par rapport à l'amplitude à vide de référence qui nous indiquerait qu'un métal est présent.

Capteur de présence

Ces capteurs seront placés aux deux extrémités du tapis afin de déterminer le départ sur le tapis et l'arrivée d'objets à la fin du tapis.

Ce capteur est imposé : référence E18-D80NK.

C'est un capteur tout ou rien NPN. Lorsqu'il capte une présence d'objet il envoie '0' sinon il envoie '1'.



Planning de réalisation du projet électronique

Nous avons réalisé un GANTT pour visualiser le chemin critique du projet (pdf joint). Les taches définies en début de projet sont les suivantes,

Taches moteur

- Effectuer un schéma puis simuler sur Matlab ;
- Etudier en détail les caractéristiques du système microcontrôleur-driver-moteur ;
- Souder la carte permettant d'utiliser le driver ;
- Tester la carte ;
- En parallèle du soudage créer le programme du microcontrôleur permettant de piloter le système ;
- Tester le système entier avec le programme du microcontrôleur.

Taches détecteur de métaux

- Déterminer les valeurs des condensateurs et bobines que l'on veut utiliser pour créer le détecteur ;
- Etudier l'implantation du détecteur sur le tapis ;
- Création des deux bobines ;
- Création des deux cartes électroniques (émission et réception) ;
- Tester les deux cartes à l'aide d'un métal pour déterminer si l'on détecte bien le métal ;
- Créer le programme permettant de déterminer la fréquence du signal sur la carte de réception ;
- Tester le système complet avec le programme.

Taches capteur de présence

- Etude des caractéristiques de fonctionnement du capteur ;
- Implantation des capteurs sur le tapis ;
- Création du programme permettant l'utilisation de ces capteurs ;
- Test du système entier (microcontrôleur + capteur).

Conclusion

Au court de ce projet nous avons rencontrés certaines difficultés qu'il nous semble important de synthétisé ici pour pouvoir éviter de les reproduire à l'avenir. Pour la partie informatique réaliser a deux nous avons choisi de réalisé le travail en binôme en alternant celui qui code et celui qui regarde pour comprendre le programme dans sa globalité. Cela nous a fait perde un peu de temps mais cela a permis d'éviter qu'un des membres du binôme soit en difficulté sur le code qu'il n'a pas développé.

Pour la partie d'électronique la réalisation d'un GANTT suggérait par notre encadrant nous a permis de découper le projet en tache successives et liés entre elles. Cela a été bénéfique et nous a permis de gagner du temps en travaillant sur des taches différentes en parallèle. Cependant il aurait était bénéfique que nous commencions plus tôt la rédaction du rapport cela nous aurais permis aux personnes qui on peut travailler sur une partie du projet ce tienne au courant au jour le jour des évolutions du projet.